

- 1) What will be the output of the following code snippet?

```
const obj1 = {Name: "Hello", Age: 16};  
const obj2 = {Name: "Hello", Age: 16};  
print(obj1 === obj2);
```

- 2) What will be the output of the following code snippet?

```
<script type="text/javascript" language="javascript">  
var x=12;  
var y=8;  
var res=eval("x+y");  
document.write(res);  
</script>
```

- 3) What will be the output of the following code snippet?

```
const obj1 = {first: 20, second: 30, first: 50};  
console.log(obj1);
```

- 4) What will be the output of the following code snippet?

```
console.log(typeof NaN);
```

- 5) console.log([] == false);

- 6) console.log(0.1 + 0.2 === 0.3);

- 7) var x = 10;

```
function test() {  
    console.log(x);  
    var x = 20;  
}
```

```
test();
```

- 8) let x = 5;

```
{  
    let x = 10;  
}  
console.log(x);
```

- 9) console.log("5" - 2);

```
console.log("5" + 2);
```

- 10) const arr = [1,2,3];

```
arr.push(4);
```

```
console.log(arr.length);
```

11) `console.log(typeof null);`

```
12) for(var i=0;i<3;i++){
    setTimeout(()=>{
        console.log(i);
    },0);
}
```

```
13) for(let i=0;i<3;i++){
    setTimeout(()=>{
        console.log(i);
    },0);
}
```

14) `console.log([] + []);`

15) `console.log(3 > 2 > 1);`

```
16) Promise.resolve()
    .then(()=>console.log("A"));
    console.log("B");
```

17) `console.log(false == "0");`

```
18) const obj = {
    name: "SurveySparrow",
    print() {
        console.log(this.name);
    }
};
```

```
const fn = obj.print;
```

```
fn();
```

19) Output `[1, 2, 3].map(num => { num * 2 })`

20) Which of the following describes the behavior of `Array.prototype.reduce()` when no initial value is provided?

- A. It defaults the accumulator to zero.
- B. It throws a `ReferenceError`.
- C. It uses the first element as the accumulator.

D.It returns undefined immediately.

21) If you use filter() on an array containing 'holes' (sparse array), what is the result for those indices?

- A. They are skipped by the callback
- B. They are included as null
- C. The method throws a RangeError
- D. They are treated as undefined

22) What is the primary difference between __proto__ and prototype in JavaScript?

23) How does Object.create(null) differ from a standard object literal {}?

24) What is the return value of document.querySelectorAll('.item')?

25) Which property should be used to change the text of an element while ignoring any HTML tags provided in the string?

26) In the DOM event flow, which phase happens first?

27) What is the purpose of the Response.json() method in the Fetch API?

28) Predict the output:

```
console.log([] == ![]);
```

29) In an arrow function, what does the this keyword refer to?

```
30) const a = {};  
    const b = {key: 'b'};  
    const c = {key: 'c'};  
    a[b] = 123;  
    a[c] = 456;  
    console.log(a[b]);
```

31) What will console.log(!'Hello'); output?

32) What is the output of console.log(NaN === NaN);?

33) What is the output of console.log(undefined == null);?

```
34) const numbers = [1, 2, 3, 4, 5];  
    const result = numbers  
        .filter(num => num % 2 === 0)
```

```
.reverse();
```

```
console.log(numbers);
```

```
35) console.log("Start");
    setTimeout(() => console.log("Timeout"), 0);
    Promise.resolve().then(() => console.log("Promise"));
    console.log("End");
```

```
36) const person1 = { name: "Alice", age: 25 };
    const person2 = Object.assign({}, person1);
    person2.name = "Bob";
    delete person1.name;
    console.log(person2.name);
```

37) Question: A developer expects this to log 0, 1, 2 with a 1-second delay between each. What actually happens, and how do you fix it?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

38) Spot the Bug: Fetch API Errors

Question: Why does the `.catch()` block fail to trigger when the API returns a 404 Not Found response?

```
fetch("https://api.example.com/missing-endpoint")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error("Request failed:", err));
```

39) You attach a single click listener to a container `<ul id="list">`. What property on the event object identifies the exact element that was clicked?

```
document.getElementById("list").addEventListener("click", (e) => {
  // Which property points to the clicked <li>?
});
```

- A) `e.currentTarget`
- B) `e.target`
- C) `e.srcParent`
- D) `e.delegateTarget`

```
40) function Animal(name) {
    this.name = name;
  }
```

```
Animal.prototype.speak = function() {
```

```
    return `${this.name} makes a noise.`;  
};  
const dog = new Animal("Rex");  
console.log(dog.hasOwnProperty("speak"));  
console.log("speak" in dog);
```

ANSWERS:

1) **False**

The strict equality operator compares objects by their references so even though the contents of both objects are the same, their references don't match resulting in false.

2) **20**

The eval() function evaluates the expression passed into it as a parameter and returns the result after evaluation.

3) **{first: 50, second: 30}**

The last declaration of the duplicate key overrides any previous ones

4) **Number**

NaN stands for Not-a-Number, but surprisingly, JavaScript considers it a value of type number.

5) **true**

Due to JavaScript type coercion, both become 0.

Type coercion in JavaScript is the automatic or implicit conversion of a value from one data type to another.

6) **False**

Floating-point precision issue.

7) **undefined**

var x is hoisted. Pass 1 (The Scanner): It scans the function for variables. It sees var x = 20; inside test(). It immediately creates a local box named x at the top of the function, but leaves it empty (undefined). It completely ignores the global x = 10 because a local one now exists. Pass 2 (The Execution): It runs the code line by line. When it hits console.log(x), it looks inside the function, finds the local x box, and prints what is inside it. Since the code hasn't reached the x = 20 line yet, the box is still empty. Therefore, it prints undefined.

8) **5**

Variables declared with let have block scope. They only exist within the closest set of curly braces {} surrounding them.

9) **3**

"52"

The subtraction (-) operator only works with numbers. JavaScript automatically converts (coerces) the string "5" into the number 5 before performing the math (5 - 2)

The addition (+) operator doubles as a string concatenation tool. If either side of the + operator is a string, JavaScript converts the other side to a string and sticks them together ("5" + "2")

10) 4

11) object

This behavior is a historical bug from the first implementation of JavaScript in 1995. In early versions, JavaScript stored values in 32-bit units composed of a type tag and a value. The type tag for objects was 000. Because null represented a null pointer (the absence of a value), its binary representation was entirely zeros. When the typeof operator read those leading zero bits, it incorrectly flagged null as an object.

12) 3

3

3

The var keyword creates a single function-scoped (or globally-scoped) variable i shared across all iterations of the loop. It does not create a new variable for each iteration. Asynchronous Execution: The setTimeout callback function is asynchronous. It gets placed into the event queue and will only execute after the synchronous for loop has completely finished running. The Final Value: By the time the loop ends, the increment condition i++ has executed a final time, leaving the shared variable i at a value of 3. When the three queued callbacks finally run, they all look up and print that same shared value.

13) 0

1

2

The variable i is declared using let. In JavaScript, let is block-scoped, which means a completely new binding (instance) of i is created for every single iteration of the for loop. Closures: When setTimeout is called, its inner callback function forms a closure over that specific iteration's version of i. Execution: Even though the setTimeout callbacks are sent to the event queue and run asynchronously after the synchronous loop finishes, each callback remembers and prints its own unique value of i (0, then 1, then 2).

14) ""

The + operator in JavaScript triggers implicit type conversion when used on non-primitive types like arrays. Array to Primitive: JavaScript attempts to convert the arrays into primitives by calling their internal valueOf() and toString() methods. String Concatenation: An empty array [] converts to an empty string "". Therefore, the expression becomes "" + "", which evaluates to an empty string ""

15) false

JavaScript's operator precedence rules, the greater-than operator (>) evaluates from left to right. `3 > 2` is evaluated first, which returns true. The expression then becomes `true > 1`. In a numeric comparison, JavaScript automatically coerces true to 1. The final comparison becomes `1 > 1`, which evaluates to false. To check if 3 is greater than 2 and 2 is greater than 1, use the logical AND operator: `(3 > 2) && (2 > 1)`

16) B A

JavaScript reads the code line by line from top to bottom. It encounters `Promise.resolve().then(...)`. Because the promise resolves immediately, its `.then()` callback is instantly scheduled to run in the Microtask. After it the scheduled callback is executed.

17) true

When using the loose equality operator (`==`), if one operand is a Boolean, JavaScript converts it to a number first. `false` becomes 0. Convert the String: The expression is now `0 == "0"`. Since one side is a number and the other is a string, JavaScript converts the string to a number. `"0"` becomes 0. Compare: The final comparison is `0 == 0`, which evaluates to true

18) undefined

`Const fn = obj.print`; this copies only the function(`print()`) not the object, this depends on how the function is called not where it was written `fn()=>` has only `print()` function. Copying `obj.print` to `fn` separates the function from the `obj` object. Global this: When you call `fn()` directly, this points to the global window object instead of `obj`. Missing Property: Because the global scope does not have a `name` property (or it is empty), it prints `undefined` in strict mode.

To fix : Use `const fn = obj.print.bind(obj)`; so this stays tied to `obj`
Use `obj.print()`;

19) [undefined, undefined, undefined]

The arrow function uses block braces without a return statement so each iteration returns `undefined` by default.

20) It uses the first element as the accumulator

Without an initial value, `reduce` skips the first element as a callback and sets it as the starting accumulator.

With initial value:

```
const arr = [10, 20, 30];
```

```
const result = arr.reduce((acc, curr) => acc + curr, 0);
```

```
console.log(result);
```

```
| Step | Accumulator | Current | Result |  
| ---- | - | - | - | - |
```


Start	0	10	10	
2	10	20	30	
3	30	30	60	

Without accumulator

```
const result = arr.reduce((acc, curr) => acc + curr);
```

Step	Accumulator	Current	Result	
----	-----	-----	-----	
Start	10	20	30	
2	30	30	60	

21) A. They are skipped by the callback

Hole => an index that does not exist in the array

```
const arr = [1, , 3];
```

Not same as

```
const arr = [1, undefined, 3];
```

```
const arr = [1, , 3];
```

```
const result = arr.filter(x => true);
```

```
console.log(result); // [1, 3]
```

22) The primary difference is that prototype is a property belonging exclusively to **constructor functions** (and classes), used to define blueprints for shared behaviors, while `__proto__` is an internal accessor property present on all object instances that points to the actual prototype object they inherit from.

23) An object created via `Object.create(null)` has **no prototype chain** (`[[Prototype]]: null`), whereas a standard object literal `{}` inherits directly from `Object.prototype`

24) The `document.querySelectorAll('.item')` method returns a static (non-live) **NodeList** object containing all document elements that match the specified CSS selector

25) textContent

26) **Capturing Phase:** The event starts at the top root/window and goes down the DOM tree to the target element. **Target Phase:** The event reaches the exact target element where it occurred. **Bubbling Phase:** The event goes back up from the target element to the root

27) It returns a promise,

reading the body is an asynchronous operation, so `json()` returns a promise that resolves to the parsed data

28) true

The ! operator converts the array to false, then comparison coerces the empty array to 0 and false to 0

29) The 'this' of the outer scope.

Arrow functions do not have their own 'this' binding; they inherit it from the context where they were defined.

30) 456

JavaScript object keys must be strings (or symbols)
When using object as a key, JS converts b to a string
String(b) // "[object Object]"
a["[object Object]"] = 123;
a[c] = 456;
a["[object Object]"] = 456;

31) true

'Hello' => true, !true => false, !false => true

32) false

NaN is not equal to anything, including itself

33) True

34) [1, 2, 3, 4, 5]

.filter() creates a new array containing [2, 4]. The subsequent .reverse() mutates that newly created array, leaving the original numbers array untouched.

35) Start → End → Promise → Timeout

Explanation:

Start and End run synchronously (Call Stack).

Promise.resolve().then(...) schedules a Microtask.

setTimeout(...) schedules a Macrotask (Task Queue).

Microtasks execute before macrotasks after the current stack empties, so Promise runs before Timeout.

36) "Bob"

Object.assign({}, person1) performs a shallow copy of person1 into a brand-new object. Updating or deleting properties on person1 afterward has zero effect on person2.

37) Logs 3, 3, 3 after 1 second because var is function-scoped. Fix: Change var to let.

38) fetch only rejects on network errors or blocked requests, not HTTP error statuses (4xx/5xx).

Explanation: The Fetch API Promise resolves successfully as long as the server responds, even with a 404 or 500 status. To catch HTTP errors, check if (!res.ok) throw new Error(...) inside the first .then().

39) **e.target** is the specific element that triggered the event (e.g., the nested or).
e.currentTarget is the element to which the event handler is attached (the).

40) false, true

hasOwnProperty("speak") checks if speak exists directly on the dog instance. Since speak lives on Animal.prototype, it returns false.

The in operator checks the instance and its prototype chain, so "speak" in dog returns true.